ESP32 Landslide Early-Warning System

Complete Wiring + Arduino IDE + Full Code + Dashboard/API + Testing Guide

Prototype note: This system monitors precursor indicators and creates configurable warnings. It cannot guarantee that a landslide will occur exactly one hour later. A validated one-hour prediction requires site-specific historical data, calibrated sensors and a validated prediction model.

1. Overall System

Sensors → ESP32 → risk calculation → LEDs/buzzer → Wi-Fi → HTTP POST → API/database → dashboard

GitHub Pages can host the dashboard frontend. It normally should not be the ESP32 POST endpoint because GitHub Pages is static hosting. Use a backend/API or database service between the ESP32 and the GitHub Pages frontend.

2. Components

Component	Qty	Purpose
ESP32 DevKit V1	1	Controller + Wi-Fi
Soil moisture sensor	1	Soil wetness
Rain sensor module	1	Rain/wetness
MPU6050	1	Tilt + acceleration
Tilt sensor/module	1	Digital tilt indication
HC-SR04	1	Distance/displacement proxy
Buzzer	1	Audible alert
Green / Yellow / Red LED	1 each	Status indication
220–330 Ω resistors	3	LED protection
1 kΩ + 2 kΩ resistors	1 each	HC-SR04 ECHO divider
Breadboard/jumpers	As needed	Wiring

3. Exact ESP32 DevKit V1 Connections

Device pin	ESP32 pin	Connection
Soil AO	GPIO 34	Analog input
Soil VCC	3.3V	Power
Soil GND	GND	Common ground
Rain AO	GPIO 35	Analog input
Rain VCC	3.3V	Power
Rain GND	GND	Common ground
MPU6050 SDA	GPIO 21	I2C SDA
MPU6050 SCL	GPIO 22	I2C SCL
MPU6050 VCC	3.3V	Power
MPU6050 GND	GND	Common ground
Tilt DO	GPIO 27	Digital input
Tilt VCC/GND	3.3V/GND	Power
HC-SR04 TRIG	GPIO 5	Digital output

Device pin	ESP32 pin	Connection
HC-SR04 ECHO	GPIO 18 via divider	Never direct 5V
HC-SR04 VCC/GND	5V/GND	Power/common ground
Buzzer + / -	GPIO 23 / GND	Active buzzer; use transistor for high-power loads
Green LED	GPIO 25 via resistor	Cathode to GND
Yellow LED	GPIO 26 via resistor	Cathode to GND
Red LED	GPIO 33 via resistor	Cathode to GND

4. HC-SR04 ECHO Voltage Divider — Mandatory

If the HC-SR04 is powered at 5 V, its ECHO output can be about 5 V. Protect the ESP32 GPIO by using this divider:

```
HC-SR04 ECHO ---- 1 kΩ ----+---- GPIO 18
                           |
                         2 kΩ
                           |
                          GND

GPIO voltage ≈ 5 × 2/(1+2) = 3.33 V
Do NOT connect ECHO directly to GPIO18.
```

All grounds must be common.

5. Arduino IDE Setup

1. Install Arduino IDE and connect the ESP32 with a USB data cable.

2. Arduino IDE → Tools → Board → Boards Manager → search **ESP32** → install the Espressif Systems package.

3. Tools → Board → ESP32 Arduino → select **ESP32 Dev Module** (or the matching DevKit V1 board).

4. Tools → Port → select the ESP32 COM port.

5. Sketch → Include Library → Manage Libraries → install **Adafruit MPU6050** and **Adafruit Unified Sensor**.

6. Paste the code in Section 6.

7. Change WIFI_SSID, WIFI_PASSWORD and DASHBOARD_URL.

8. Verify, then Upload. If upload waits, hold BOOT briefly during upload.

9. Open Serial Monitor at **115200 baud**.

6. Complete ESP32 Arduino Code

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <math.h>

// ===== USER SETTINGS =====
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// This must be an API endpoint, NOT a normal GitHub Pages URL.
const char* DASHBOARD_URL =
  "http://YOUR_SERVER_IP:5000/api/sensor-data";

const char* API_KEY = "";   // Optional X-API-Key

// ===== PINS =====
#define SOIL_PIN 34
#define RAIN_PIN 35
#define SDA_PIN 21
#define SCL_PIN 22
#define TILT_PIN 27
#define TRIG_PIN 5
#define ECHO_PIN 18
#define BUZZER_PIN 23
#define GREEN_LED 25
#define YELLOW_LED 26
```

```
#define RED_LED 33

// ===== CALIBRATION =====
// Measure your actual sensor values and change these.
const int SOIL_DRY = 4095;
const int SOIL_WET = 1500;
const int RAIN_DRY = 4095;
const int RAIN_WET = 1000;
const int TILT_ACTIVE = LOW;

// ===== TIMING =====
const unsigned long SAMPLE_MS = 10000UL;       // 10 seconds
const unsigned long SEND_MS   = 10000UL;       // 10 seconds
const unsigned long WIFI_MS   = 15000UL;

// 360 x 10 sec = about 1 hour of history.
const int HISTORY_SIZE = 360;

Adafruit_MPU6050 mpu;
bool mpuOK = false;

int soilRaw=0, rainRaw=0;
float soilPct=0, rainPct=0;
float angleX=0, angleY=0, accel=0;
float distanceCm=-1, baselineDistance=0, displacementCm=0;
bool tiltActive=false, sensorFault=false;

float soilHist[HISTORY_SIZE], rainHist[HISTORY_SIZE];
int histIndex=0, histCount=0;
float soilTrend=0, rainTrend=0;
int riskScore=0;

unsigned long lastSample=0, lastSend=0, lastWiFi=0;

float pctFromRaw(int raw, int dry, int wet) {
  float p = 100.0 * (raw - dry) / (float)(wet - dry);
  return constrain(p, 0, 100);
}

void connectWiFi() {
  if (WiFi.status() == WL_CONNECTED) return;
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("WiFi: ");
  unsigned long start=millis();
  while(WiFi.status()!=WL_CONNECTED && millis()-start<10000) {
    delay(500); Serial.print(".");
  }
  Serial.println();
  if(WiFi.status()==WL_CONNECTED) {
    Serial.print("IP: "); Serial.println(WiFi.localIP());
  } else Serial.println("WiFi connection failed");
}

float readUltrasonic() {
  digitalWrite(TRIG_PIN,LOW); delayMicroseconds(2);
  digitalWrite(TRIG_PIN,HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN,LOW);
  long us=pulseIn(ECHO_PIN,HIGH,30000);
  if(us==0) return -1;
  return us*0.0343/2.0;
}

void readMPU() {
  if(!mpuOK) { angleX=angleY=accel=0; return; }
  sensors_event_t a,g,temp;
  mpu.getEvent(&a,&g,&temp);
  accel=sqrt(a.acceleration.x*a.acceleration.x+
             a.acceleration.y*a.acceleration.y+
             a.acceleration.z*a.acceleration.z);
  angleX=atan2(a.acceleration.y,a.acceleration.z)*180.0/PI;
  angleY=atan2(-a.acceleration.x,
               sqrt(a.acceleration.y*a.acceleration.y+
                    a.acceleration.z*a.acceleration.z))*180.0/PI;
}

void updateHistory() {
  soilHist[histIndex]=soilPct;
  rainHist[histIndex]=rainPct;
  histIndex=(histIndex+1)%HISTORY_SIZE;
  if(histCount<HISTORY_SIZE) histCount++;
}

float trend(float a[]) {
  if(histCount<2) return 0;
  int old=(histIndex-histCount+HISTORY_SIZE)%HISTORY_SIZE;
  int newest=(histIndex-1+HISTORY_SIZE)%HISTORY_SIZE;
  return a[newest]-a[old];
}

int calculateRisk() {
  int s=0;
  if(soilPct>=80)s+=3; else if(soilPct>=60)s+=2; else if(soilPct>=40)s++;
  if(rainPct>=80)s+=3; else if(rainPct>=60)s+=2; else if(rainPct>=40)s++;
```

```
  float tilt=max(abs(angleX),abs(angleY));
  if(tilt>=20)s+=3; else if(tilt>=10)s+=2; else if(tilt>=5)s++;

  if(tiltActive)s+=2;
  if(accel>13)s+=2; else if(accel>11)s++;

  if(baselineDistance>0) {
    if(displacementCm>5)s+=3;
    else if(displacementCm>2)s+=2;
    else if(displacementCm>1)s++;
  }

  if(histCount>=60) {
    if(soilTrend>25)s+=2; else if(soilTrend>10)s++;
    if(rainTrend>25)s+=2; else if(rainTrend>10)s++;
  }
  return s;
}

String statusText() {
  if(sensorFault)return "SENSOR_FAULT";
  if(riskScore>=10)return "DANGER";
  if(riskScore>=7)return "HIGH_RISK";
  if(riskScore>=4)return "WARNING";
  return "SAFE";
}

void warningOutput() {
  String st=statusText();
  digitalWrite(GREEN_LED,LOW);
  digitalWrite(YELLOW_LED,LOW);
  digitalWrite(RED_LED,LOW);
  digitalWrite(BUZZER_PIN,LOW);

  if(st=="SAFE") digitalWrite(GREEN_LED,HIGH);
  else if(st=="WARNING") {
    digitalWrite(YELLOW_LED,HIGH);
    if((millis()/1000)%5==0) digitalWrite(BUZZER_PIN,HIGH);
  }
  else if(st=="HIGH_RISK") {
    digitalWrite(RED_LED,HIGH);
    if((millis()/1000)%2==0) digitalWrite(BUZZER_PIN,HIGH);
  }
  else if(st=="DANGER") {
    digitalWrite(RED_LED,HIGH);
    digitalWrite(BUZZER_PIN,HIGH);
  }
  else digitalWrite(YELLOW_LED,HIGH);
}

void readAll() {
  soilRaw=analogRead(SOIL_PIN);
  rainRaw=analogRead(RAIN_PIN);
  soilPct=pctFromRaw(soilRaw,SOIL_DRY,SOIL_WET);
  rainPct=pctFromRaw(rainRaw,RAIN_DRY,RAIN_WET);

  readMPU();

  distanceCm=readUltrasonic();
  if(distanceCm>0) {
    if(baselineDistance<=0) baselineDistance=distanceCm;
    displacementCm=abs(distanceCm-baselineDistance);
  }

  tiltActive=(digitalRead(TILT_PIN)==TILT_ACTIVE);

  // Treat missing ultrasonic or MPU as a fault for this prototype.
  sensorFault=(distanceCm<0 || !mpuOK);

  updateHistory();
  soilTrend=trend(soilHist);
  rainTrend=trend(rainHist);
  riskScore=calculateRisk();
  warningOutput();

  Serial.println("===== SENSOR DATA =====");
  Serial.printf("Soil %.1f%% raw %d\n",soilPct,soilRaw);
  Serial.printf("Rain %.1f%% raw %d\n",rainPct,rainRaw);
  Serial.printf("Tilt X %.2f Y %.2f\n",angleX,angleY);
  Serial.printf("Accel %.2f m/s2\n",accel);
  Serial.printf("Distance %.2f cm, displacement %.2f cm\n",distanceCm,displacementCm);
  Serial.printf("Soil trend %.2f, rain trend %.2f\n",soilTrend,rainTrend);
  Serial.printf("Risk %d -> %s\n",riskScore,statusText().c_str());
}

void sendDashboard() {
  if(WiFi.status()!=WL_CONNECTED) return;

  HTTPClient http;
  String url=DASHBOARD_URL;

  if(url.startsWith("https://")) {
    WiFiClientSecure secure;
    secure.setInsecure(); // Prototype only; validate certificates in production.
```

```
    http.begin(secure,url);
  } else {
    WiFiClient client;
    http.begin(client,url);
  }

  http.addHeader("Content-Type","application/json");
  if(String(API_KEY).length()>0) http.addHeader("X-API-Key",API_KEY);

  String json="{";
  json+="\"device\":\"ESP32_DEVKIT_V1\",";
  json+="\"soil_moisture\":"+String(soilPct,2)+",";
  json+="\"rain_level\":"+String(rainPct,2)+",";
  json+="\"soil_raw\":"+String(soilRaw)+",";
  json+="\"rain_raw\":"+String(rainRaw)+",";
  json+="\"tilt_x\":"+String(angleX,2)+",";
  json+="\"tilt_y\":"+String(angleY,2)+",";
  json+="\"acceleration\":"+String(accel,2)+",";
  json+="\"distance_cm\":"+String(distanceCm,2)+",";
  json+="\"displacement_cm\":"+String(displacementCm,2)+",";
  json+="\"tilt_sensor\":"+String(tiltActive?"true":"false")+",";
  json+="\"soil_trend\":"+String(soilTrend,2)+",";
  json+="\"rain_trend\":"+String(rainTrend,2)+",";
  json+="\"history_samples\":"+String(histCount)+",";
  json+="\"risk_score\":"+String(riskScore)+",";
  json+="\"status\":\""+statusText()+"\",";
  json+="\"sensor_fault\":"+String(sensorFault?"true":"false")+",";
  json+="\"uptime_ms\":"+String(millis());
  json+="}";

  Serial.println("POST: "+json);
  int httpCode=http.POST(json);
  Serial.printf("HTTP code: %d\n",httpCode);
  if(httpCode>0) Serial.println(http.getString());
  http.end();
}

void setup() {
  Serial.begin(115200);

  pinMode(TILT_PIN,INPUT);
  pinMode(TRIG_PIN,OUTPUT);
  pinMode(ECHO_PIN,INPUT);
  pinMode(BUZZER_PIN,OUTPUT);
  pinMode(GREEN_LED,OUTPUT);
  pinMode(YELLOW_LED,OUTPUT);
  pinMode(RED_LED,OUTPUT);

  Wire.begin(SDA_PIN,SCL_PIN);

  if(mpu.begin()) {
    mpuOK=true;
    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
    Serial.println("MPU6050 detected");
  } else Serial.println("MPU6050 NOT detected");

  connectWiFi();
  Serial.println("System ready");
}

void loop() {
  unsigned long now=millis();

  if(WiFi.status()!=WL_CONNECTED && now-lastWiFi>=WIFI_MS) {
    lastWiFi=now;
    connectWiFi();
  }

  if(now-lastSample>=SAMPLE_MS) {
    lastSample=now;
    readAll();
  }

  if(now-lastSend>=SEND_MS) {
    lastSend=now;
    sendDashboard();
  }

  warningOutput();
}
```

7. What You Must Change Before Upload

```
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

const char* DASHBOARD_URL =
  "http://YOUR_SERVER_IP:5000/api/sensor-data";
```

Never publish real Wi-Fi passwords, API keys, database passwords or GitHub tokens in a public GitHub repository.

8. GitHub Dashboard Setup

If your GitHub site is something like **https://yourname.github.io/landslide-dashboard/**, use it as the dashboard frontend. The ESP32 should send data to an API/database.

```
ESP32
  |
  | POST JSON
  v
Backend / API / Database
  |
  | GET / read data
  v
GitHub Pages
  |
  v
Live dashboard
```

1. Put HTML/CSS/JavaScript dashboard files in your GitHub repository.

2. Enable GitHub Pages for the repository if you want GitHub to host the frontend.

3. Create/deploy an API endpoint such as **/api/sensor-data** using a backend or database service.

4. Make the API accept POST requests containing the JSON fields from the ESP32.

5. Store the latest reading and historical readings.

6. Make the GitHub Pages JavaScript fetch the readings and update cards/charts.

7. Use HTTPS for the deployed API.

8. Copy the deployed API endpoint into DASHBOARD_URL in the ESP32 sketch.

Do not put a GitHub Personal Access Token in ESP32 firmware just to update files in a repository. Use a backend/database API.

9. Example JSON

```
{
  "device":"ESP32_DEVKIT_V1",
  "soil_moisture":72.50,
  "rain_level":64.20,
  "soil_raw":2100,
  "rain_raw":1800,
  "tilt_x":4.25,
  "tilt_y":2.10,
  "acceleration":9.81,
  "distance_cm":48.20,
  "displacement_cm":1.40,
  "tilt_sensor":false,
  "soil_trend":12.20,
  "rain_trend":18.50,
  "history_samples":120,
  "risk_score":5,
  "status":"WARNING",
  "sensor_fault":false,
  "uptime_ms":1234567
}
```

10. Risk Levels

Score	Status	Indication
0–3	SAFE	Green LED; buzzer off
4–6	WARNING	Yellow LED; periodic short beep
7–9	HIGH_RISK	Red LED; repeated beeps
10+	DANGER	Red LED; continuous buzzer

These thresholds are illustrative prototype thresholds. They must be calibrated and validated before any real safety use.

11. One-Hour History

The code stores 360 samples. At one sample every 10 seconds, that is approximately one hour. It calculates long-term changes in soil moisture and rain level. This is a trend monitor, not a guaranteed one-hour prediction.

For stronger prediction research, collect long-term field data and include rainfall accumulation, soil moisture rate-of-change, tilt rate, displacement rate, acceleration anomalies, slope/geology information and confirmed event labels.

12. First Test Procedure

1. Power OFF and inspect every wire.

2. Confirm all grounds are common.

3. Confirm HC-SR04 ECHO passes through the 1 kΩ/2 kΩ divider.

4. Connect USB and open Serial Monitor at 115200.

5. Confirm MPU6050 detected.

6. Confirm Wi-Fi connected and IP printed.

7. Check soil and rain raw values.

8. Move the MPU6050 slowly; check tilt values.

9. Move the HC-SR04 target; check distance.

10. Trigger the tilt sensor; check its digital state.

11. Check green LED in normal condition.

12. Test controlled wetness/rain input and observe risk score.

13. Test the API only after local sensors work.

14. Confirm HTTP response code is successful and dashboard updates.

13. Calibration

Record actual raw values from your sensors. Update these constants:

```
const int SOIL_DRY = 4095;
const int SOIL_WET = 1500;
const int RAIN_DRY = 4095;
const int RAIN_WET = 1000;
const int TILT_ACTIVE = LOW;
```

For soil: record dry and very-wet raw values. For rain: record dry and wet values. Sensor polarity differs between modules, so do not assume these numbers are universal.

14. Troubleshooting

Problem	Check
Upload fails	Correct ESP32 board, COM port, USB data cable; press BOOT during upload.
MPU6050 missing	3.3V, GND, SDA=21, SCL=22; check I2C address.
HC-SR04 reads -1/0	TRIG=5, ECHO divider to GPIO18, common GND, target distance.
Buzzer always on	Read risk score/status in Serial Monitor; check thresholds and sensor readings.
Soil/rain percentage backwards	Measure actual raw dry/wet values and update calibration constants.
HTTP POST fails	Check Wi-Fi, API URL, endpoint path, server status and HTTPS configuration.
GitHub dashboard not updating	GitHub Pages is frontend only; connect it to the API/database.
ESP32 resets	Use stable power; do not drive a large buzzer directly from GPIO; use a transistor/driver.

15. Final Implementation Checklist

1. Wire all sensors exactly according to the pin table.

2. Add the HC-SR04 voltage divider.

3. Install ESP32 board support and Adafruit libraries.

4. Enter Wi-Fi credentials locally.

5. Upload the complete sketch.

6. Open Serial Monitor at 115200.

7. Test every sensor.

8. Calibrate soil/rain values.

9. Check LEDs and buzzer.

10. Deploy an API/backend/database.

11. Put the API URL in DASHBOARD_URL.

12. Verify HTTP POST response.

13. Connect the GitHub Pages dashboard to the API/database.

14. Test the complete chain from sensor to dashboard.

15. Only then perform controlled outdoor testing.

16. Important Safety / Engineering Limitation

This is an educational/research prototype, not a certified landslide warning device. A reliable field system needs validated thresholds, redundant sensors, robust power and communication, environmental protection, site-specific calibration, false-alarm analysis, historical data and a defined emergency response procedure.

17. Next Step for Your GitHub Project

If you send the GitHub repository link, the repository can be checked to determine whether it contains only the dashboard frontend or also an API/backend. The ESP32 DASHBOARD_URL and dashboard JavaScript can then be matched to the exact project structure.